DefectAI Business Plan and Codebase Audit

Validation-first AI inspection for packaging defects

Generated from repository audit and the staged business plan.

Executive status: DefectAI is close to the validation-first MVP described in the business plan. The strongest implemented pieces are workspace/project separation, batch upload, queued YOLO inference, structured prediction storage, simple feedback, CSV export, and dashboard/report UI. The highest-priority gaps are PDF reporting, buyer-facing report quality, stronger model traceability, review queue, approved examples, and manual retraining preparation.

Business Plan Interpretation

Main Product Vision

Build a paid validation product for packaging defect inspection. Customers upload real images, AI predicts defects, operators give simple feedback, and the system produces buyer-facing validation evidence before the customer invests in production automation.

MVP Boundary

The smallest sellable workflow is: **batch upload -> AI prediction -> simple feedback -> validation report**.

Build Now

- Customer/workspace/project setup
- Batch image upload
- YOLO inference
- Structured prediction storage
- One-click feedback
- Feedback-backed CSV/PDF validation report
- Fast review screens and paid pilot readiness

Build Next

- Review queue
- Approved examples
- Manual retraining export/preparation
- Dataset and model version metadata

Delay

- Camera integration and edge deployment
- Kubernetes, microservices, and full MLOps
- Automatic retraining and automatic model promotion
- Full annotation editor and enterprise features

Current Implementation Summary

The codebase is a React/Vite frontend, FastAPI backend, MySQL database, Redis/RQ queue, local file storage, and YOLO worker. This matches the plan's recommended simple architecture.

- Backend models include workspaces, memberships, invitations, users, projects, inspection batches, detection jobs, detection boxes, and human feedback.
- Backend routes cover auth, workspace/project management, single image upload, batch upload, detection jobs, feedback, batch reports, and CSV export.
- Frontend supports dashboard upload/review, batch upload, recent batches, batch detail, job review, and batch report pages.
- YOLO inference stores detections and annotated images, with Redis/RQ handling asynchronous work.

Done / Missing / Delay

Done

- Workspace/project/user flow
- Batch and single-image upload
- Redis/RQ worker pipeline
- YOLO inference and annotated image serving
- Detection boxes persisted
- Job and batch status tracking
- Simple feedback types: correct, false positive, wrong class, missed defect, not sure, bad image
- Batch summary and CSV export
- Dashboard, batch detail, job review, and report pages
- Role-based workspace permissions

Missing

- PDF report export
- Buyer-ready validation report sections
- Review queue
- Approved examples
- Manual retraining export from approved examples
- Dataset version table
- Real model version table
- Explicit packaging-focused defect taxonomy/project setup
- Pilot readiness polish: report language, storage cleanup, audit trail, demo dataset flow

Delay

- Camera integration
- Edge deployment
- Kubernetes
- Automatic retraining
- Full annotation editor
- Enterprise SSO
- Broad multi-industry platform features

- Full MLOps/model registry UI

Gap Analysis

Requirement	Status	Related Files	Missing Pieces	Risk / Concern	Priority
Workspace/customer separation	Done	`workspace.py`, `workspaces.py`, workspace frontend	Business naming still uses workspace, not customer/account	Good enough for MVP	High
Project/pilot setup	Done	`project.py`, `workspaces.py`, admin/project pages	Pilot metadata, defect taxonomy, paid pilot status	Projects are generic	High
Batch upload	Done	`batches.py`, `upload_service.py`, `BatchUploadPanel.tsx`	Upload limits, duplicate handling, batch pre-validation	Large batches may stress local storage/RQ	High
AI inference	Done	`tasks.py`, `detection_pipeline_service.py`, `yolo_detector.py`	Explicit model version registry and provenance	`best.pt` is implicit and gitignored	High
Structured prediction storage	Partially done	`DetectionBox`, `DetectionJob`, repository persistence	Dedicated AI prediction naming, reliable model version	Traceability weak if multiple models are used	High
Prediction review screen	Done	`ImageViewer.tsx`, `DetectionCard.tsx`, `JobReviewPage.tsx`	Review queue navigation	Good MVP coverage	High
Simple feedback buttons	Done	`DetectionCard.tsx`, `JobReviewPage.tsx`, feedback API	Consistent dashboard vs job-detail UX	Dashboard feedback is job-level; detail supports box-level	High
Feedback storage	Done	`HumanFeedback`, `detection.py`, repository	Audit history if needed	Latest-only behavior loses feedback history	High
CSV report export	Done	`reports.py`, `BatchReportPage.tsx`	Polished buyer-facing formatting	Useful but not yet a polished deliverable	High
PDF report export	Not implemented	None	PDF endpoint, template, frontend download	Business plan says PDF/CSV is must-have	High
Validation report summary	Partially done	`reports.py`, report schemas, report page	Operational value, ROI/time saved, confidence distribution, next steps	Current report is technical, not buyer-ready	High
Review queue	Not implemented	Batch/job pages approximate review	`review_tasks` model, routes, queue UI, escalation rules	Important next stage, not first blocker	Medium
Approved examples	Not implemented	None	`approved_annotations`, expert approval UI, export	Needed before retraining, not before first pilot	Medium
Manual retraining preparation	Not implemented	`ml/scripts/convert_voc_to_yolo.py`	YOLO export from approved annotations, training docs	Do not automate yet	Medium

Requirement	Status	Related Files	Missing Pieces	Risk / Concern	Priority
Dataset versions	Should be delayed	None	`dataset_versions`, frozen snapshots	Later than review/approved examples	Low
Model versions	Partially done	`DetectionJob.model_name`, `model_version`	`model_versions` table, active model per project	Nullable strings are not enough for comparison	Medium
Paid pilot readiness	Partially done	Workspace/project/batch/report flow	PDF report, report language, cleanup, audit trail, demo flow	Technically close but not sales-ready	High

Phased Implementation Roadmap

Phase 1: Stabilize Current Validation MVP

Goal: make the existing batch/prediction/feedback/report flow reliable enough for demos and paid diagnostics.

- **Frontend:** keep stable 3-column dashboard, improve empty/loading/error states, make upload/review navigation predictable.
- **Backend:** add tests around batch upload, job completion, feedback, and reports.
- **Database:** no new tables.
- **API:** keep current routes.
- **Acceptance:** upload 50+ images, process jobs, submit feedback, export CSV without manual DB fixes.
- **Do not build:** review queue, retraining, model registry.

Phase 2: Structured Prediction And Feedback Storage

- **Goal:** make prediction/feedback data trustworthy for reports.
- **Frontend:** make dashboard feedback clearly job-level; keep detailed feedback in job review.
- **Backend:** populate model name/version on completion; test latest feedback behavior.
- **Database:** avoid new version tables unless necessary now.
- **Acceptance:** every prediction row is traceable to model metadata and feedback.
- **Do not build:** model registry UI.

Phase 3: Review Workflow And Operator Feedback Improvement

- **Goal:** give operators a fast review path without annotation burden.
- **Frontend:** add reviewed/unreviewed filters and next/previous image navigation.
- **Backend:** add reviewed/unreviewed job filters.
- **Database:** no review task table yet unless simple filters are insufficient.
- **Acceptance:** operators can review a batch quickly with one-click feedback.
- **Do not build:** expert approval or annotation editor.

Phase 4: Validation Report PDF/CSV

- **Goal:** produce a buyer-facing deliverable.
- **Frontend:** add Export PDF beside CSV; improve report language.
- **Backend:** add PDF generator service and confidence/operational summary fields.
- **Database:** no new tables unless storing report snapshots becomes necessary.

- **API:** add `GET /workspaces/{workspace_id}/projects/{project_id}/batches/{batch_id}/export.pdf`.
- **Acceptance:** founder can send a PDF report to a pilot prospect.
- **Do not build:** automated ROI calculator beyond simple report text.

Phase 5: Review Queue And Approved Examples

- **Goal:** separate raw feedback from training-quality examples.
- **Frontend:** add review queue page and simple expert approval form.
- **Backend:** add review task and approved annotation services/routes.
- **Database:** add `review_tasks` and `approved_annotations`.
- **Acceptance:** raw feedback never directly becomes training data.
- **Do not build:** full annotation editor.

Phase 6: Manual Retraining Preparation

- **Goal:** make offline retraining possible without automating it.
- **Frontend:** add export approved examples action.
- **Backend:** add YOLO export service and manual training docs.
- **Acceptance:** approved examples can train a customer-specific YOLO model manually.
- **Do not build:** automatic retraining or promotion.

Phase 7: Dataset/Model Versioning

- **Goal:** track frozen training sets and trained models after approved examples exist.
- **Backend/DB:** add `dataset_versions`, `model_versions`, and active model selection per project.
- **Frontend:** minimal read-only model/version info.
- **Acceptance:** reports can compare model A vs model B.
- **Do not build:** full MLOps UI.

Phase 8: Production/Pilot Hardening

- **Goal:** make paid pilots safe and repeatable.
- **Tasks:** audit logs, backups, storage cleanup, stronger file validation, seed/demo data, onboarding checklist.
- **Acceptance:** run 2-3 paid pilots without babysitting every step.
- **Do not build:** enterprise SSO, camera/edge, Kubernetes.

Next 5 Concrete Coding Tasks

1. Add backend tests for batch upload, report summary, and CSV using current models and fake jobs.
2. Ensure `model_name` and `model_version` are populated when a job completes.
3. Improve `BatchReportPage` and `reports.py` with buyer-facing sections: dataset summary, AI output, feedback analysis, performance recommendation, and next steps.
4. Add PDF export endpoint and frontend PDF download button.
5. Add reviewed/unreviewed filtering and next-image workflow for batch review.

Architectural Concerns

- `DetectionBox` effectively serves as `ai_predictions`, but missing model-version linkage will become confusing. Add explicit model metadata before pilots.
- Raw feedback is collapsed to latest per job/box. This is good for simple reports but weak for auditability.
- `model_version` is a nullable string, not a real model version entity. Acceptable now, but fix before retraining comparisons.
- Batch upload enqueues one RQ job per image and stores files locally. Fine for pilots, but add file limits and cleanup before large datasets.
- UI copy should shift from broad industrial platform language toward packaging inspection validation and paid pilot reports.
- The report is the buying artifact. Do not let review queue, approved examples, or retraining distract from report quality.

Scope rule: every new feature should help close a paid pilot, improve the validation report, or reduce operator effort. Otherwise delay it.